Introduction to Data Engineering: A Comprehensive Overview

Lecture Notes for a 4-Hour Session - Course 1, Week 1

Based on materials by DeepLearning.AI and the textbook "Fundamentals of Data Engineering" by J Matt Housley

January 17, 2026

Contents

1 Part 1: The Landscape of Data Engineering

1.1 Introduction: What is Data Engineering?

Data Engineering is the backbone of the modern data-driven organization. Initially, data generated by software applications was often seen as a mere byproduct or "digital exhaust"—a byproduct useful mainly for debugging. As organizations began to recognize the immense intrinsic value locked within this data, a new discipline emerged, tasked with building robust systems to refine this raw material into actionable insights.

A formal definition, from the course's companion textbook "Fundamentals of Data Engineering" by Reis and Housley, provides a comprehensive view:

> "Data engineering is the development, implementation, and maintenance of systems and processes that take in raw data and produce high-quality, consistent information that supports downstream use cases, such as analysis and machine learning. Data engineering is the intersection of security, data management, DataOps, data architecture, orchestration, and software engineering."

In simpler terms, your job as a data engineer is to get raw data from somewhere, turn it into something useful, and then make it available for downstream use cases.

1.2 The Data Engineering Lifecycle

The Data Engineering Lifecycle is a conceptual framework that models the flow of data from its origin to the point of value delivery. It consists of five primary stages, with storage as a foundational element supporting the core processes.

Generation This is the origin point of all data. It occurs in various **source systems** which are typically not owned or controlled by the data engineer. Examples include:

- Application Databases (OLTP systems)
- Logs from servers and applications
- Events from mobile and web applications
- Data from IoT sensors
- Files from third-party vendors (e.g., CSVs, JSON)

Storage This foundational stage involves persisting data. Storage is not a single step but a continuous process; data is stored after ingestion, between transformations, and before being served. Key considerations include the storage format (e.g., Parquet, Avro), the physical medium (HDD, SSD), and the system (e.g., Object Storage like Amazon S3, or a relational database).

Ingestion The process of moving data from its source system into a centralized storage layer, such as a data lake or data warehouse. This is often one of the most significant bottlenecks, as it involves interacting with systems outside of the data engineer's direct control.

Transformation This is where raw data is converted into a valuable asset. It is the "T" in ETL (Extract, Transform, Load) or ELT (Extract, Load, Transform). Activities include:

- **Cleaning:** Handling nulls, correcting data types, removing duplicates.
- **Normalizing/Denormalizing:** Structuring the data according to a specific data model.
- **Enriching:** Joining the data with other datasets to add context.
- **Aggregating:** Summarizing data to create business-level metrics.

Serving The final stage where the processed, high-quality data is made available to consumers. The "serving layer" can be a data warehouse, a data mart, or an API that exposes data to users like:

- **Analytics and BI:** Enabling analysts to create dashboards and reports.
- **Machine Learning:** Providing feature sets for data scientists to train models.
- **Reverse ETL:** Feeding enriched data (e.g., customer lead scores) back into operational source systems like a CRM.

The end-to-end flow of data through these stages is often referred to as a **data pipeline**.

1.3 Lifecycle Undercurrents

Underpinning the entire lifecycle are several foundational practices, or "undercurrents." These are not sequential stages but cross-cutting concerns that must be addressed throughout.

- **Security:** Ensuring data confidentiality, integrity, and availability at every stage. This includes access control, encryption, and network security.
- **Data Management:** A broad discipline that includes data governance (policies and standards), data quality, and metadata management (maintaining data about data).
- **DataOps:** Applying DevOps principles to data. It focuses on automation, continuous integration/continuous delivery (CI/CD), testing, and monitoring to improve the quality and reliability of data products.
- **Data Architecture:** The high-level blueprint for the data systems. It defines the structure, components, and interactions that fulfill the business's long-term data strategy.
- **Orchestration:** The process of scheduling, coordinating, and managing complex, interdependent data workflows to ensure they run reliably and efficiently.
- **Software Engineering:** The use of coding best practices, version control, and testing methodologies to build data pipelines as robust, maintainable software.

2 Part 2: The Evolution of the Field and the Engineer's Role

2.1 A Brief History of Data Engineering

The role and tools of a data engineer have evolved dramatically over the past few decades in response to technological shifts and the increasing scale of data.

1970s-1980s The invention of the **relational database** and **SQL** provided the tools for structured data storage. **Bill Inmon** then conceptualized the **data warehouse**, separating analytical workloads from operational databases to improve performance and enable historical analysis.

1990s Ralph Kimball advanced data warehousing with **data modeling** techniques like the star schema for business intelligence (BI). The mainstream adoption of the **internet** created a new wave of "web-first" companies and, with them, an unprecedented explosion in user-generated data.

The 2000s: The "Big Data" Era The sheer **Volume, Velocity, and Variety** (the 3 Vs) of data from companies like Google and Yahoo broke traditional data warehousing paradigms.

- Google's internal challenges led to landmark papers on its internal technologies, including **MapReduce** (2004), a novel programming model for processing massive datasets in a distributed manner.
- This inspired the open-source community, particularly at Yahoo, to create **Hadoop** (2006), an open-source framework that democratized big data processing and gave rise to the "Big Data Engineer."
- Concurrently, **Amazon Web Services (AWS)** launched its public cloud, offering pay-as-you-go access to scalable compute (EC2) and storage (S3), fundamentally changing how systems are built and deployed.

The 2010s-Present: The Modern Data Stack The operational complexity of managing Hadoop clusters led to a new wave of innovation. The industry shifted towards **abstract, simplified, and managed services**, primarily on the cloud.

- The "Modern Data Stack" emerged, characterized by modular, cloud-native, best-of-breed tools. For example, using Fivetran for ingestion, Snowflake for storage, dbt for transformation, and Looker for BI.
- The focus moved from managing low-level infrastructure to integrating these powerful services. As a result, the role of the data engineer has moved "up the value chain," and the distinction of "big data" has faded—it's all just data engineering now.

2.2 The Data Engineer and Stakeholders

A data engineer is a crucial liaison between the technical and business sides of an organization. Understanding the perspectives of different stakeholders is essential for success.

Upstream Stakeholders These are the data producers, primarily **Software Engineers**, who build the operational systems that generate data. Collaboration is key to ensure data is generated in a clean, well-structured format and to communicate changes that might break downstream pipelines.

Downstream Stakeholders These are the data consumers who use the data products you build. This diverse group includes:

- **Data Analysts** who create dashboards and reports to analyze business trends.
- **Data Scientists** who build predictive models.
- **ML Engineers** who specialize in deploying, monitoring, and maintaining machine learning models in production environments.
- **Business Leaders** (e.g., Executives, Marketing, Sales) who use insights for strategic decision-making.

2.3 Data Maturity and the Engineer's Role

As a company's use of data evolves, so does the role of the data engineer. We can think of this evolution in three stages of **data maturity**:

1. **Stage 1: Starting with Data.** The company has just begun its data journey. Goals may be fuzzy. The data engineer is often a generalist, playing multiple roles (analyst, scientist, engineer) to get quick wins and demonstrate the value of data. The focus is on building a solid foundation.

2. **Stage 2: Scaling with Data.** The company has formal data practices and is moving away from ad-hoc requests. The challenge is now to build scalable, robust data architectures. Data engineering roles become more specialized. The focus is on creating repeatable processes and robust systems.

3. **Stage 3: Leading with Data.** The company is data-driven. Data is seamlessly integrated, and self-service analytics and ML are common. The data engineer's role becomes highly specialized, focusing on automation, building custom tools, and managing "enterprisey" concerns like governance and advanced DataOps.

3 Part 3: From Strategy to Systems

3.1 Delivering Business Value

The most important principle for a data engineer is to focus on delivering **business value**. Technology is a tool, not the goal. As data warehouse pioneer Bill Inmon famously advised, when asked for advice for newcomers, he likened it to a bank robber's philosophy: "Go to where the money is." For a data engineer, this means focusing on projects that have a clear and measurable impact on the business.

Value is ultimately defined by your stakeholders. It's crucial to understand their goals and how your work helps them succeed. This perception of value is what secures funding, earns trust, and builds a successful data culture.

3.2 The Requirements Gathering Process

To build a valuable system, a data engineer must be skilled at translating high-level business goals into specific, actionable technical requirements. This is a critical discovery process.

The Hierarchy of Requirements

1. **Business Requirements:** The high-level goals of the business (e.g., "reduce customer churn by 5%").
2. **Stakeholder Requirements:** Needs of specific users (e.g., "the customer success team needs a daily list of users at high risk of churning").
3. **System Requirements:** The technical specification for the system, which are divided into:
   - **Functional (The "WHAT"):** What the system does (e.g., "The system will generate a churn score for every active user").
   - **Non-Functional (The "HOW"):** The qualities of the system (e.g., "The churn scores must be updated every 24 hours with a data latency of no more than 4 hours"). This includes performance, reliability, scalability, and security constraints.

Key Elements of Requirements Gathering

1. Learn what existing systems and solutions are in place.
2. Understand the pain points and problems with the current solutions.
3. Ask what **actions** stakeholders plan to take with the data. This is the most crucial question. It uncovers the true goal behind a request. A need for "real-time data" might actually mean "a daily report to decide which customers to call," which has very different technical implications.
4. Identify any other stakeholders you need to talk to.

3.3 Choosing Tools and Technologies

Once requirements are clear, the next step is selecting the right tools. This is not about picking the trendiest technology, but about making strategic trade-offs. Key considerations from the textbook include:

- **Team Size and Capabilities:** Does your team have the skills for a complex, code-heavy tool, or would a low-code/managed service be more effective?
- **Build vs. Buy:** Does building a custom solution provide a unique competitive advantage, or is an off-the-shelf tool a better use of resources? Always lean towards buying or using open-source unless building is a core differentiator.
- **Monolith vs. Modular:** A monolithic system (like a traditional all-in-one ETL tool) is simple to reason about but can be brittle and hard to change. A modular, "best-of-breed" architecture is flexible and scalable but introduces complexity in orchestration and interoperability.
- **Cost Optimization (FinOps):** Understand the total cost of ownership (TCO) and the total *opportunity* cost of ownership (TOCO)—the value you lose by being locked into one technology and unable to adopt a better one later. FinOps is the practice of bringing financial accountability to the cloud, managing spending through collaboration between engineering and finance.

4 Part 4: Data Engineering in Practice on the Cloud

Modern data engineering is, for the most part, cloud-native. This course adopts a **cloud-first** perspective, focusing on the skills and platforms you are most likely to encounter in the industry today, with a **just-in-time** approach to learning specific tools.

4.1 Cloud Deployment Models

- **On-Premises:** The company owns and operates its own data centers. This involves significant upfront capital expenditure (CapEx) and operational overhead.
- **Cloud:** The company rents resources from a public provider (AWS, GCP, Azure). This is an operational expenditure (OpEx) model that offers elasticity and pay-as-you-go pricing.
- **Hybrid:** A strategic combination of both on-premises and cloud resources, often used during migrations or to meet specific regulatory needs.

4.2 Core Cloud Concepts and Services

Understanding the fundamental building blocks of the cloud is essential.

IT Resources The on-demand services you use.

- **Compute:** Provides the power to run code. Examples include **Amazon EC2** (virtual machines), **AWS Lambda** (serverless functions), and container services like **Amazon EKS** (Kubernetes).
- **Storage:** Provides a place to persist data. It's crucial to understand the different types:
  - **Object Storage** (e.g., **Amazon S3**): The backbone of the data lake. Stores files (objects) and is incredibly scalable and durable. It is not a filesystem and is accessed via API.
  - **Block Storage** (e.g., **Amazon EBS**): A virtual hard drive for an EC2 instance. It offers the low-latency performance needed for transactional databases.
  - **File Storage** (e.g., **Amazon EFS**): A network-attached file system that can be shared across multiple compute instances.
- **Databases:** Managed services for various database engines, from relational (**Amazon RDS**) to data warehouses (**Amazon Redshift**).
- **Networking:** Services like **Amazon VPC** allow you to create logically isolated private networks in the cloud to secure your resources.

Global Infrastructure

- **Regions:** A physical geographic location in the world where a cloud provider has data centers (e.g., US East, Europe-Frankfurt).
- **Availability Zones (AZs):** Each Region consists of multiple, physically separate, and isolated AZs. Each AZ has independent power, cooling, and networking. This design enables high-availability and fault-tolerant architectures.

4.3 Security: The Shared Responsibility Model

Security in the cloud is a partnership. It is crucial to understand where the cloud provider's responsibility ends and yours begins. The "high-rise apartment" analogy is useful here: the building owner is responsible for the building's security (locks, secure foundation), but the tenant is responsible for locking their own door.

Cloud Provider's Responsibility (Security OF the Cloud) The provider is responsible for securing the physical infrastructure: data centers, hardware, and the virtualization layer that separates tenants.

Your Responsibility (Security IN the Cloud) You are responsible for everything you build on top of that infrastructure. This includes:

- Securely configuring AWS services.

- Managing identity and access (using the **Principle of Least Privilege**).
- Patching guest operating systems on your VMs.
- Setting up firewalls and network access controls.
- Encrypting your application data.

5 Week 1 Summary

This comprehensive introduction has provided a high-level framework for data engineering. The key takeaways for building a successful career are:

1. **Focus on Value:** Always start by understanding stakeholder needs and aligning your work with clear business goals.
2. **Be Systematic:** Translate those needs into concrete functional and non-functional requirements before choosing tools.
3. **Build for Evolution:** Use a flexible, iterative approach. The systems you build today must be able to evolve with the business and technology landscape of tomorrow.

By embracing this holistic and value-driven mindset, a data engineer becomes not just a builder of pipelines, but a critical driver of business success.